SystemVerilog 검증 수업 상세 정리

2026.11.09 (월) · 1~6교시 · 카운터 랜덤 제약 리뷰 → RAM 설계 → RAM 테스트벤치 v2/v3
출처: `1109class.txt`(녹음 전사), `tb_counter_rsm(_v2).sv`, `ram.sv`, `tb_ram(_v2/_v3).sv`, xsim 실행 로그

> **오늘 수업을 관통한 말.** "답이 1번만 있는 게 아니라 여러 개가 있다. 여러분이 플랜을 짜서 그대로 만들어 보고, 테스트하고, 검증해 보라는 거다." 구조 자체보다 **왜 그 시점에 드라이브·샘플·판정했는지 타이밍으로 설명할 수 있는가**가 핵심이다.

꼭 챙길 일정	날짜	내용
핸드라이팅 시험	11/16 (월)	2시간, 개념 + 설계 코딩 (2-3절)
UVM 검증 발표	11/23 (월)	기존 설계를 UVM으로 검증, 시나리오·흐름·결과 발표 (2-1절)

구분	주요 내용	녹음 행
1교시	카운터 숙제 리뷰: 유지 횟수 랜덤화, Driver 반복, rand/dist/randc, 발표 주제 논의	1~63
2교시	발표 방식·일정·시험·로드맵·면접 조언, SRAM 설계, 검증 시나리오, Expected 저장 구조	69~149
3교시	판정 시점 결정, X 처리, 블록 다이어그램, 이벤트 레이스	154~198
4교시	Expected를 만드는 두 구조, 조합/순차 차이, TB 코딩 순서와 타입 선택	204~233
5교시	Scoreboard 작성, 실행·디버그, 리포트, 주소 randc, 넌블로킹 이벤트	240~273
6교시	Generator→Scoreboard 메일박스 추가, 매 사이클 판정, 숙제	279~296
7장	tb_ram 3개 버전 변경점, 컴포넌트별 해설, 실제 로그 타임라인, 보완할 점	코드·로그
8~10장	강사 질문 기반 복습 문제, 숙제 목록, 녹음 불명확 항목	전체

보충 녹음에 없고 정리하며 덧붙인 설명 확인 작성된 코드를 직접 실행·대조한 결과 회색 숫자: `1109class.txt`의 행 번호 인용문: 전사 오류를 문맥에 맞게 다듬은 것

1교시 · 카운터 숙제 리뷰: 3사이클 이상 연속 동작

1-1. 문제 제기: 왜 기존 풀이는 랜덤 제약이 아닌가 1~6행

과제는 "run/stop이 3사이클 이상 연속되는 시나리오"였다. 많은 풀이가 카운터로 3번을 세면서 이전 값을 유지했는데, 강사의 지적은 이렇다.

> "3 이상 카운터를 임의로… 그건 랜덤 컨스트레인트가 아니잖아. 3 이상 발생시키려면 변수 하나 만들어서, 3보다 크고 어떤 값보다 작은 랜덤 수를 만들어 내야죠. 그게 뭐가 되는 거야? 반복 횟수가 되는 거죠."

비교	이전 풀이 (`tb_counter_rsm.sv`)	수업 후 (`tb_counter_rsm_v2.sv`)
유지 방법	Generator에서 `cnt`와 `prv_*`로 직전 값을 고정	transaction의 `run_stop_hold`로 반복 횟수를 전달
유지 길이	항상 3으로 고정	3~16 범위에서 매번 랜덤
랜덤화	`randomize() with {if(cnt>1)…}` 조건 필요	`randomize()`만 호출, 제약은 클래스 안에 선언

비교	이전 풀이 (`tb_counter_rsm.sv`)	수업 후 (`tb_counter_rsm_v2.sv`)
반복 담당	Generator	Driver

1-2. 무엇을 기준으로 잡을까 7~10, 15행

강사 질문	수업 결론
run/stop과 mode 중 무엇을 기준?	카운터의 동작을 생각하면 **run_stop**. 기준 없이 뜬금없이 만들 수는 없다.
둘 다 hold를 만들면 안 되나?	가능하다(run_stop용, mode용 각각). 다만 복잡하니 하나씩 한다.
hold 범위는?	처음엔 3~256으로 언급. 256은 "거의 빡빡"해 여러 번 바뀌는 걸 볼 수 없어 **3~16**으로 변경 23~24행

1-3. 반복을 누가 맡을까: Generator vs Driver 10~19행

학생들 머릿속에 "Gen→Drv→Mon→Scb를 한 바퀴 도는 것"만 있어서 생긴 혼동을 먼저 짚었다. 실제 프로세스는 두 묶음으로 나뉜다.

프로세스 묶음	동기화 수단	특징
Generator ↔ Driver	메일박스 + 이벤트	서로 맞물려 움직인다
Monitor → Scoreboard	메일박스	상승 엣지에 맞춰 혼자 계속 돈다 (카운터 TB의 경우)

안	동작	판단
Generator 반복	hold를 확인하고 같은 메시지를 hold번 put, 돌아가서 대기	가능하지만 조건이 피곤해지고, 그 사이 Driver 상태를 알 수 없다
Driver 반복	메시지 1개에 hold를 실어 보내면 Driver가 그만큼 반복 후 이벤트 전송	**채택.** Generator는 1회 생성만 하면 되고 제어가 단순해진다

핵심 문장: "보내고 다 끝났는지 확인하고 또 가면 돼요. 트랜잭션 아이템에 홀드 카운트를 추가하면 되죠." 18행

1-4. 변수와 제약 설계 23~33, 41~42행

```
// tb_counter_rsm_v2.sv:39~76 발췌
rand  bit reset;
randc bit run_stop;                  // randc 쓰면 5:5, 쓰면 dist 조작이 안된다.
rand  bit mode;
rand  bit clear;
rand  int unsigned run_stop_hold;    // 음수가 나오면 안되므로 unsigned

constraint clear_dist { clear dist { 0:/ 9, 1:/ 1 }; }   // reset, mode도 동일
constraint run_stop_hold_range { run_stop_hold inside {[3:16]}; }
```

변수	설정	이유
`run_stop_hold`	`inside {[3:16]}`, unsigned	반복 횟수이므로 음수 불가. 최소 3을 주고, 상한으로 무한 유지 방지

변수	설정	이유
`clear`	0:9, 1:1	별도 제약이 없으니 설계자가 정함. 자주 걸면 카운트를 볼 수 없어 "가끔"
`mode`	강제 또는 확률	"업 모드만 볼 거야" 또는 "1:1로 볼 거야" 등 목적에 따라 결정 25행
`run_stop`	`randc`	1:1이면 충분하므로 사이클릭 사용

> **보충** 코드의 `:/`는 범위 전체에 가중치를 나누는 연산자이고, `:=`는 각 값에 같은 가중치를 준다. `0`, `1`처럼 단일 값에서는 결과가 같고, `[0:9]` 같은 범위를 쓸 때 차이가 난다.

1-5. rand, dist, randc 34~40행

구분	`rand + dist`	`randc` (cyclic)
동작	지정한 가중치로 값 선택	범위의 모든 값이 한 번씩 나올 때까지 중복 없이 순환
1비트	비율 자유 조절	2번 중 0과 1이 각 1번 → 5:5
4비트 예시	가중치대로 편중 가능	16번 안에 각 값이 1번씩
주의	횟수가 적으면 비율 오차	**`dist`와 같이 쓸 수 없음**. 값이 반드시 한 번씩 나와야 하므로 확률 조작이 불가

강사 경험담: 100번 돌려도 정확히 50:50이 나오지 않을 때가 있고, 횟수를 늘리면 비슷해진다. (원인은 7-5절 ③ 참고)

1-6. Driver 구현과 하강 엣지 함정 44~47행

처음 작성은 "negedge → 메시지 가져오기 → 드라이브"를 그대로 repeat했다. 이러면 메시지를 가져올 때마다 negedge를 한 번 더 기다리게 되어 **매 메시지마다 1클럭씩 쉰다**. 해결은 첫 드라이브와 나머지 반복을 분리하는 것이다.

```
// tb_counter_rsm_v2.sv:171~196 발췌
forever begin
    gen2drv_mbox.get(tr);
    // 첫 드라이브: mode, reset, clear, run_stop 인가
    repeat(tr.run_stop_hold-1) begin
        @(negedge cnt_vif.clk);
        // 같은 값 다시 인가
    end
    ->gen_next_evt;
end
```

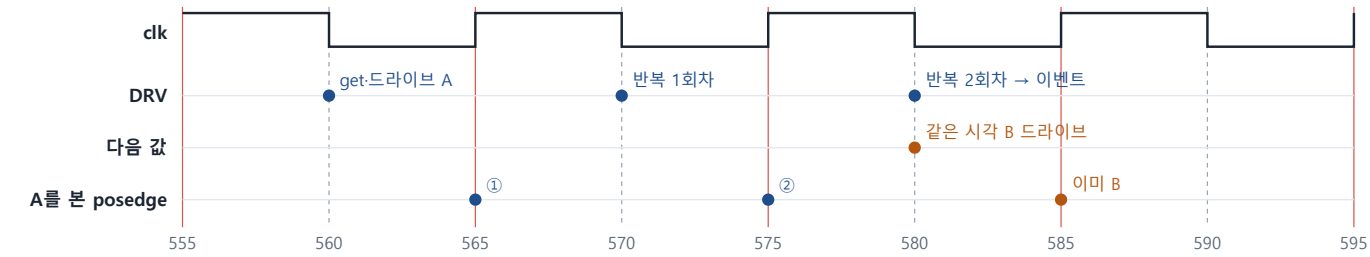
1-7. 결과 확인과 발표 요구사항 48~53행

hold 값이 나오는지 `debug_print`에 추가해 확인했고, 수업 화면에서는 8, 8, 14, 16, 13 순으로 나왔다. 이 시나리오는 run/stop만 제어하고 mode는 "혼자 놀고" 있으니 mode까지 확장해 보라고 했다.

> **강사가 요구한 결과물.** ① 무엇을 랜덤으로 만들지 시나리오 ② 어떻게 제어할지 흐름도 ③ "이렇게 제어하고 이렇게 동작시켰어요" 설명 ④ 시뮬레이션 결과와 통계 리포트

확인 **실제 유지 길이는 hold-1 클럭이다.** seed 1 로그의 hold 값은 수업 화면과 같은 8, 8, 14, 16, 13이었지만, 다음 DRV까지의 간격은 70, 70, 130, 150, 120ns로 (hold-1)×10ns와 전 구간 일치했다. `hold=3`이면 2클럭만 유지되어 **"3사이클 이상" 조건을 만족하지 못한다.** 마지막 반복의 negedge에서 이벤트를 보내면, 같은 시각에 다음 메시지가 바로 드라이브되어 값이 덮이기 때문이다.

tb_counter_rsm_v2: hold=3인 트랜잭션



tb_counter_rsm_v2 seed 1의 실제 구간: hold=3인 560ns 트랜잭션이 580ns에 끝난다

2교시 · 일정 공지와 SRAM 설계

2-1. UVM 발표 방식 54~60, 69~78, 96행

항목	내용
언어	UVM. "SV로 하고 또 UVM 하는 의미가 없다"며 SV 발표는 하지 않음
설계	기존 구현을 그대로 사용, **검증 과정만** 새로 구현
주제 후보 ①	10,000진 카운터: FSM + 데이터패스. 버튼 디바운스는 시간이 오래 걸려 제외, FND는 제외 쪽으로 논의
주제 후보 ②	UART + FIFO + ASCII 디코더. 수신 문자에 따라 run/stop 등을 동작시키는 형태로 언급
결정 상태	"2가지 안을 놓고 가" 단계로, 최종안은 녹음에서 확정되지 않음
발표일	11/23 (월). 11/16은 UVM까지 하기에 시간이 부족하다고 판단

2-2. 앞으로의 일정 74~82, 88~94행

시기	내용
11/9~10	SV로 RAM, FIFO. "오늘 내일 다 끝날 것 같다"
11/11~13	UVM 전에 Synopsys 툴 학습 + SV 추가 실습 *
11/16 (월)	핸드라이팅 시험
11/16 주	UVM 실습: adder → 플립플롭 → 카운터 → RAM → FIFO → (가능하면) UART
11/23 (월)	UVM 검증 발표
11/23 이후	RISC-V RV32I 약 3~4주: 버스, GPIO·UART·FND 컨트롤러, FPGA에서 SW 실행, UVM 검증
12월~1월	C언어·임베디드. 복잡하지 않게, 어셈블리로 변환해 설명하는 방식
2월~3월	영상 처리·AI 등 플러스 알파, 최종 팀 프로젝트(약 4주)
3월 말	최종 발표. 우수 2~3팀은 기업 대상 외부 발표 예정(계획은 변동 가능)

2-3. 11/16 시험 99~111행

항목	내용
시간·형식	2시간으로 끊음. 시간이 부족할 것이라고 안내. 개념 문제 + 설계 코딩 문제
개념	SV라고 해서 문법 시험을 따로 보지는 않음. **시뮬레이션 레이스 컨디션**은 나올 수 있음
설계	복잡하지 않게: MUX, 디코더, adder(하프→풀 조립식 제외), ALU, 비교기, 카운터, 플립플롭, FSM
나올 수 있음	FIFO, UART, ASM을 코드로 옮기는 문제. 범위는 FIFO·UART까지
제공 정보	UART 스펙·타이밍은 문제에 제시. 복잡한 인터페이스는 내지 않음

2-4. 강사 조언 19~23, 83~95, 113~115, 176, 191행

주제	내용
숙제	못 했으면 한 데까지 제출. 어떻게 생각했는지 알아야 오해를 바로잡아 줄 수 있다.
합격 사례	비전공 출신 반장이 한 달 고생하다 SV에서 감을 잡음. 비결은 "제일 먼저 해보고, 시키는 거 다 하고, 발표 자원". 가만히 있으면 아무도 모른다.
면접	내가 기술한 내용은 전부 설명할 수 있어야 한다. "왜 했어? 그건 또 왜 했어?"가 계속될 때 "다른 친구가 해서 몰라요"는 꽝.
준비	검증·설계 어느 직무를 지원하든 포트폴리오 설명 + 전공 질문 대비
학습 태도	"수능 문제 풀러 온 게 아니라 트러블슈팅하러 온 것." AI가 알려 주는 답 말고 먼저 고민할 것. 틀려도 된다.
최종 프로젝트	주제·스펙은 팀이 정하고, 혼자서는 범위가 작다. 4주는 짧으니 팀원과 협업이 중요.

2-5. SRAM 설계 116~131행

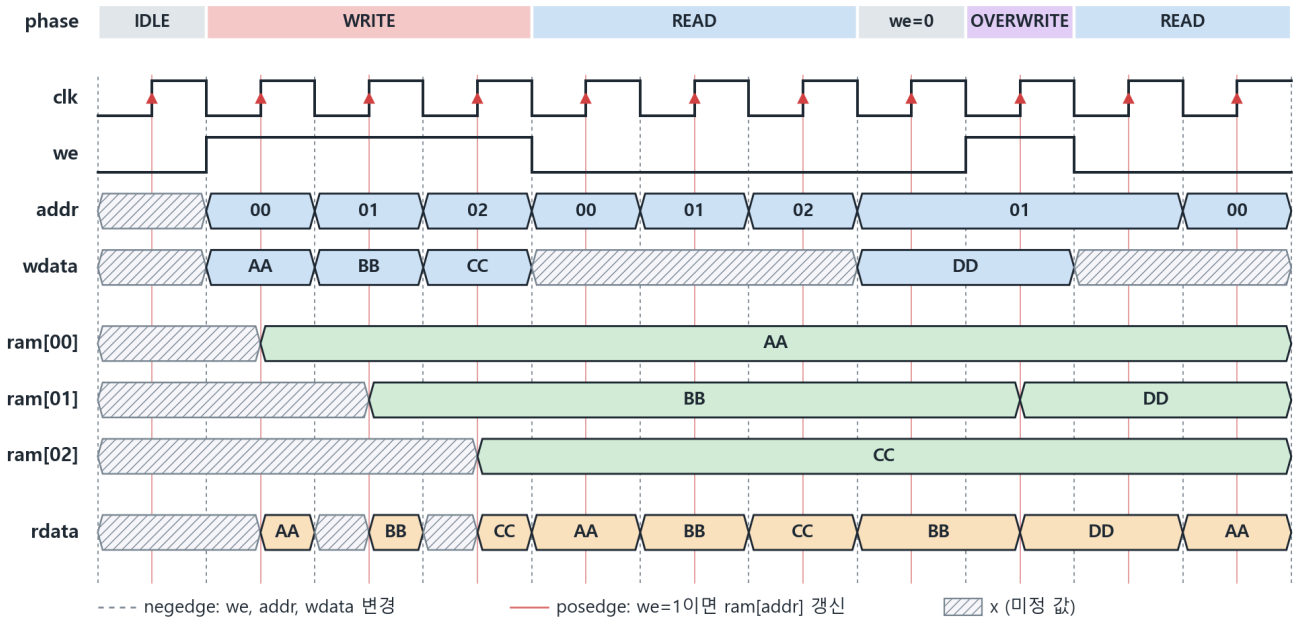
항목	내용
주소 공간	6비트 → 2^6 = 64칸 (주소 0~63). 데이터 폭 8비트
리셋	없음 (SRAM의 특징)
쓰기	we=1일 때 clk 상승 엣지에서 지정 번지에 저장
읽기	주소 지정만으로 출력. 방식에 따라 **조합 출력**(주소가 바뀌면 바로) / **순차 출력**(상승 엣지 이후)
오늘 범위	조합 출력부터 구현·검증. 순차 출력도 해 봐야 함

```
// ram.sv:13~33 발췌
logic [7:0] ram [0:63];

always_ff @(posedge clk) begin
    if (we) ram[addr] <= wdata;     // 동기 쓰기
    // else rdata <= ram[addr];      // 순차 출력일 때
end

assign rdata = ram[addr];           // 조합 출력
```

RAM 64x8 타이밍도 (동기 쓰기 / 조합 읽기)



ram_timing.png: 입력은 하강 엣지에 변경, 저장은 상승 엣지, rdata는 addr와 메모리 변화에 즉시 반응. 쓰기 직후 rdata에 새 값이 보이는 점이 6교시 내용과 이어진다.

2-6. 검증 시나리오와 랜덤 대상 133~136행

구분	내용
시나리오 ①	모든 주소에 대해 쓰기/읽기 검사
시나리오 ②	we=1일 때만 메모리가 바뀌는지 검사
시나리오 ③	조합 출력이 주소 변경에 바로 반응하는지 검사
랜덤 대상	`addr`, `wdata`, `we` 모두. we는 제약 없는 1비트이고 wdata도 비트별 불량을 보려면 랜덤이 필요

2-7. 드라이브·샘플·저장 구조 논의 136~146행

강사 질문	수업 진행
언제 드라이브할까?	지금까지처럼 하강 엣지
언제 샘플할까?	일단 상승 엣지마다. 샘플 타이밍은 Monitor가 정하고, Scoreboard는 받은 값으로 판정한다
Expected는 어디에?	"뭔가 가지고 있어야 되잖아" → Scoreboard에 RAM과 같은 크기의 배열을 하나 둠
큐를 쓰면 안 되나?	큐는 FIFO 성격이라 주소로 접근하는 용도엔 불편. 안 되는 건 아니지만 배열이 간단
언제 값을 저장?	we=1이면 DUT와 같은 번지에 같은 값을 저장

3교시 · 판정 시점, X 처리, 이벤트 레이스

3-1. 쓰기 순간에 Pass/Fail을 볼 수 있는가 146~148, 162~168행

we=1인 상승 엣지 순간에는 메모리가 아직 갱신 전이라 애매하다는 문제 제기에서 출발했다. 강사는 안 된다고 한 것이 아니라 "어떻게 판단할 거냐"를 물은 것이라고 정정했다.

질문	결론
쓸 때마다 저장된 값을 판정해야 하나?	꼭 그럴 필요는 없다. 버퍼에 같이 써 두었으니 **나중에 읽을 때 비교**하면 잘 썼는지까지 검사된다
그럼 언제 판정?	시나리오 1: **we=0일 때만** 판정, we=1이면 버퍼에 저장만
모니터는 we=0일 때만?	아니다. we=1일 때도 올려야 Scoreboard가 Expected를 저장할 수 있다 185, 189~190행
왜 계속 모니터링?	wdata도 we도 랜덤이라 언제 쓸지 예측 불가. 카운터처럼 다음 값을 계산할 수 없다

3-2. 아직 안 쓴 주소를 읽으면 168~171행

랜덤 주소라 한 번도 안 쓴 번지를 읽을 수 있고, 이때 DUT의 rdata는 X다. Expected 배열 타입과 비교 연산자의 조합별 결과는 다음과 같다.

Expected 타입 / 비교	미기록 주소 값	판정	의미
`byte` (2-state) / `===`	Expected 0, rdata X	FAIL	0을 쓴 경우와 구분되지 않고 정상 동작이 FAIL 처리
`logic` / `==` (`tb_ram.sv`)	X == X → X	if 거짓 → FAIL	정상 동작이 FAIL 처리
`logic` / `===` (v2, v3)	X === X → 참	PASS	**수업 결론**

> "바이트는 2상태니까 안 되지. 4상태, 로직으로 잡아서 케이스 등가로 하면 X값 처리가 되잖아요."

3-3. 블록 다이어그램을 먼저 그려라 172~176행

Generator에서 랜덤 생성 → Driver로 전달 → get 후 드라이브까지가 첫 하강 엣지다. "그 다음은 어떻게 할지 플랜을 잡으라"는 것이 실습 주문이었고, DUT와 Interface부터 시작해 블록을 직접 그려 보라고 했다.

3-4. 이벤트를 같은 시각에 날리면 178~194행

학생 안: "하강 엣지에서 드라이브가 끝나면 Monitor로 이벤트를 준다." 강사의 반문은 "같은 시간인데 받을 수 있어요?"였다.

순서	같은 시각에 벌어지는 일	결과
정상	Monitor가 먼저 `@(ev)` 대기 → Driver가 `->ev`	이벤트 수신
레이스	Driver가 먼저 `->ev` → 그 뒤에 Monitor가 `@(ev)` 진입	이벤트를 놓치고 다음 이벤트까지 멈춤

회피 방법	동작	언급 / 사용
`#1` 후 이벤트	시간차를 두어 레이스 자체를 없앰	"제일 좋은 건 1나노 지나서 날리는 게 편하겠죠" 218행

회피 방법	동작	언급 / 사용
`->>ev`	NBA 영역에서 이벤트 발생	268~270행 · `tb_ram_v2`
`wait(ev.triggered)`	같은 time step에 이미 발생했어도 감지	268행 · `tb_ram_v2`
구조로 해결	샘플 시점을 분리해 이벤트 없이 처리, 또는 이벤트·메일박스 추가	193~195행

3-5. 방법은 하나가 아니다 195~196행

> "이제 답안지 정답지 찾지 마시고, 주관식이야. 답이 1번만 있는 게 아니라 여러 개가 있는데, 여러분이 플랜을 짜서 그 플랜대로 만들어 보시고 테스트해 보시고 검증해 보시라고."

4교시 · Expected를 만드는 두 구조와 코딩

4-1. 샘플 시점 정리 204~206행

DUT는 we=1이면 상승 엣지에 저장한다. 샘플은 상승 엣지, 하강 엣지, 또는 그 사이의 특정 시간 중 어디든 가능하다. **드라이브 이후여야 한다는 것만 확실하고**, 정확한 시점은 설계자가 결정한다.

4-2. 두 가지 구조 206~214행

비교	A. Scoreboard 버퍼 방식	B. Generator 전달 방식
Expected 출처	Monitor가 올린 wdata를 배열에 저장	Generator가 메일박스로 Scoreboard에 직접 전달
판정 시점	we=0일 때만	매 사이클 가능. 단 쓰기는 상승 엣지 이후 샘플 필요
구성	메일박스 2개 + 이벤트 2개	메일박스 3개 + 이벤트
주의	"저장됐을 거라고 감안"하고 읽을 때 간접 확인	타이밍 주의사항이 생김 ("1피코라도 지나야")
작성 파일	`tb_ram_v2.sv`	`tb_ram_v3.sv`

A 구조의 흐름: Generator가 생성·put → Driver가 get 후 하강 엣지에 드라이브 → 이벤트로 Monitor를 깨움 → Monitor가 상승 엣지에 샘플 → Scoreboard가 저장 또는 판정 → Generator로 `gen_next_evt`.

4-3. 조합 출력과 순차 출력은 시나리오가 다르다 215~217행

드라이버는 하강 엣지에 드라이브하고, 조합 출력은 주소가 바뀌면 데이터도 곧바로 바뀌므로 상승 엣지 샘플 시나리오가 성립한다. 순차 출력은 상승 엣지 이후에 값이 나오므로 같은 시나리오를 그대로 쓸 수 없다.

4-4. UVM과의 관계 220~222행

이벤트 제어는 UVM에서는 하지 않아도 되며 조건만 잘 걸면 된다. 지금은 문제가 생기면 분석해서 원인을 찾는 과정 자체를 연습하는 단계다.

4-5. 코딩 순서와 타입 선택 223~233행

순서	내용
① **interface**	`clk`를 입력으로 받고 `addr`, `we`, `wdata`, `rdata` 선언

순서	내용
② **transaction**	rand 변수 + `rdata`, `debug_print`
③ **generator**	메일박스 + `gen_next_evt`
④ **driver**	메일박스 + virtual interface + `mon_next_evt`
⑤ **monitor**	메일박스(`mon2scb_mbox`) + virtual interface + `mon_next_evt`
⑥ **scoreboard**	메일박스 + `gen_next_evt`
⑦ **environment·top**	메일박스와 이벤트를 생성해 전달, `fork…join_any`

신호	타입	이유
`addr`, `wdata`, `we`	`rand bit`	입력은 0/1 두 상태만 테스트하면 충분. X 입력을 테스트하려면 그때 `logic`
`rdata`	`logic`	출력의 X를 관찰해야 하므로 4-state

5교시 · Scoreboard 작성과 디버그

5-1. Monitor에 이벤트가 꼭 필요한가 240~241행

학생 질문: "이벤트 없이 상승 엣지에서 계속 보내면 안 되나?" 일단 이벤트 없이 상승 엣지만 기다려 올려 보고, 실행 결과로 판단하기로 했다. 뒤의 디버그에서 필요성이 드러난다.

5-2. Scoreboard 로직 243~249행

조건	동작
we=1	판정 안 함. `expected_rdata[addr]`에 wdata 저장
we=0	Expected와 rdata를 비교해 Pass/Fail 집계
공통	처리 후 `gen_next_evt`를 Generator로 전송

5-3. 디버그 기록 250~267행

증상	원인 / 대응
빨간 줄(컴파일 오류)	`randomize` 철자 오타
PASS가 X 위주로 나옴	쓴 번지를 다시 읽을 확률이 낮음 → 실행 횟수를 늘려서 확인 (100회)
Driver가 한 칸씩 밀림	Monitor가 이벤트를 안 기다리고 상승 엣지에 먼저 처리. `mon_next_evt` 대기를 추가해 해결
이벤트 추가 후 정상 동작	"잘 됐네. 위험한 건데." 우연히 맞았을 수 있으니 `->>`와 `wait(.triggered)`로 명확히 표현할 것을 안내
리포트 필요	Scoreboard에 `report` task와 카운터 추가, Environment에서 호출

5-4. 주소에도 randc? 259~260행

주소에 `randc`를 쓰면 64개를 중복 없이 한 번씩 사용할 수 있지만 "100% 보장은 아니다"라고 했다. we처럼 1:1이면 되는 신호에는 randc가 편리하다.

5-5. 넌블로킹 이벤트 268~271, 279행

일반 이벤트 `->`는 블로킹으로, 타임슬롯이 시작될 때 바로 발생한다. 화살표 머리가 두 개인 `->>`는 NBA 영역에서 발생한다. 레이스가 생기지 않도록 "자기한테 편한 것을 쓰면 된다"고 정리했다.

> **보충** **한 time step의 처리 순서.** Active(블로킹 대입, `->`) → Inactive(`#0`) → NBA(`<=`, `->>`) → … → Postponed. `.triggered`는 해당 time step이 끝날 때까지 참으로 유지되므로, 이벤트가 대기보다 먼저 발생해도 감지할 수 있다.

6교시 · 구조 변경과 숙제

6-1. Generator → Scoreboard 메일박스 281~284행

변경	내용
메일박스	Generator가 Scoreboard로도 트랜잭션을 전달 (1개 추가)
이벤트	필요. 없으면 "제너레이터 혼자 막 돌아버린다." 기다리는 구조는 동일
Scoreboard	메일박스 2개 모두 get
효과	we=1일 때도 Pass/Fail 판정 가능. 이전엔 조건을 맞추기 어려워 못 했을 뿐

6-2. 샘플 타이밍이 바뀌는 이유 286~288행

상승 엣지 순간의 값은 DUT 저장 전이다. 쓰기 결과까지 판정하려면 상승 엣지에서 샘플하면 안 되고, **다음 하강 엣지에 샘플하거나 상승 엣지 뒤 1ns 지연**을 두어야 한다. `tb_ram_v3`는 지연 방식을 택했다.

6-3. 같은 트랜잭션을 두 곳에 보내도 되나 291~292행

학생 질문: 같은 트랜잭션을 놓고 보내면 Driver가 먼저 가져가서 문제가 생기지 않나? 강사 답: "TR2를 하나 생성해서 보내는 게 안전하다. 하나로 해도 되긴 하더라, 전에 해 보니까. 근데 위험하죠."

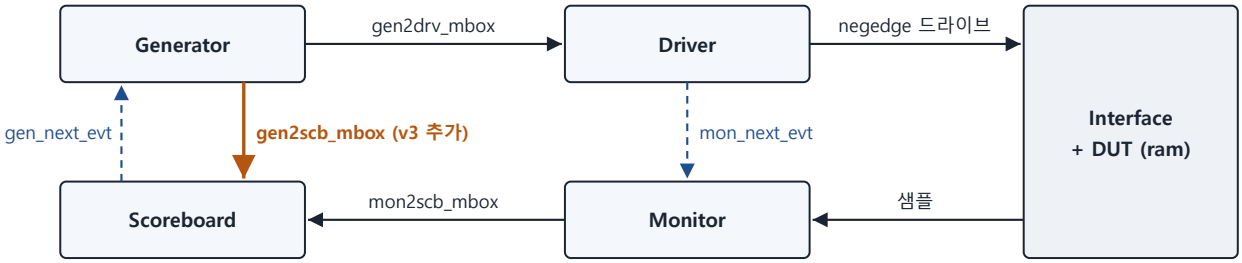
6-4. 숙제 289~293행

메일박스 하나 추가 + 이벤트는 각자 생각대로 + 매 사이클 Pass/Fail 판정. 시나리오를 짜고 앞의 구조와 비교한다.

> "검증이 별거 없어요. 로직을 잘 이해하고 있으면 '이런 타이밍에 맞춰서 하면 되겠구나' 하고 만들면 되고, 모르면 헤매다 시간만 가는 거죠."

7장 · 작성 코드 정리와 실행 결과

7-1. 전체 구조



실선: 메일박스/신호, 파선: 이벤트, 주황: v3에서 추가된 경로

7-2. 버전별 변경점

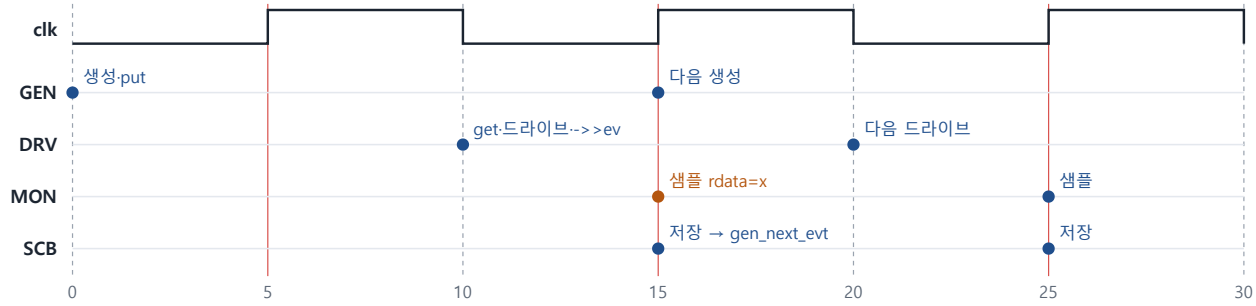
항목	`tb_ram.sv`	`tb_ram_v2.sv`	`tb_ram_v3.sv`
Generator	vif 보유, negedge 대기 후 생성	vif 제거, 이벤트로만 진행	v2 + gen2scb put
반복·종료	`while(enable)`, `join_none`, `#300`	`repeat(run_count)`, `join_any`	v2와 동일
Drv→Mon 이벤트	`->` + `@`	`->>` + `wait(.triggered)`	`->` + `@`
샘플 시점	posedge	posedge	posedge + `#1`
메일박스	2개	2개	3개
we=1 처리	Expected 저장	Expected 저장	wdata와 rdata 비교 판정 + 저장
we=0 비교	`==`	`===`	`===`
`rdata`	`rand logic`	`logic`	`logic`
리포트	pass/fail, write 집계 (total·read는 미집계)	total·read 집계 추가	read·write 각각 pass/fail 추가

7-3. tb_ram_v3 컴포넌트별 해설

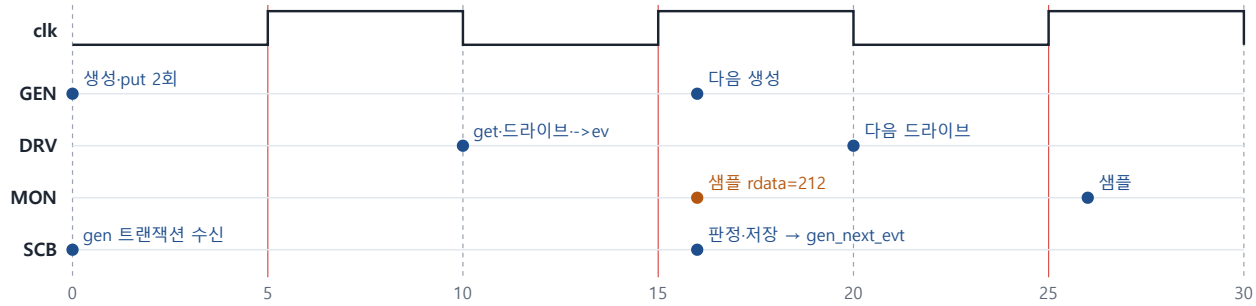
컴포넌트	행	역할과 흐름
transaction	22~52	`addr`·`wdata`는 rand, `we`는 randc, `rdata`는 샘플값 저장용 logic
generator	79~90	new → randomize → 두 메일박스에 put → `gen_next_evt` 대기, run_count번 반복
driver	121~132	negedge → get → we·addr·wdata 드라이브 → `mon_next_evt` 발생
monitor	162~178	이벤트 대기 → posedge → `#1` → 인터페이스 값으로 새 트랜잭션 구성 → put
scoreboard	222~278	gen·mon 메일박스 get → we에 따라 판정·저장 → `gen_next_evt`
environment	337~350	4개 프로세스를 fork, Generator가 끝나면 report 후 `disable fork`
tb_ram (top)	360~392	clk 10ns 주기 생성, DUT 연결, `env.run(300)`

7-4. 실제 로그 타임라인 (seed 1, 첫 두 트랜잭션)

tb_ram_v2: posedge에서 샘플



tb_ram_v3: posedge + 1ns에서 샘플



세로 빨간선: posedge, 회색 점선: negedge. 두 버전 모두 트랜잭션 하나에 1클럭(10ns)이 걸린다.

```
// tb_ram_v2 (posedge 샘플, 저장 전)
10ns :[DRV] addr = 16 ,we = 1, wdata = 212 , rdata=   x
15ns :[MON] addr = 16 ,we = 1, wdata = 212 , rdata=   x

// tb_ram_v3 (posedge + 1ns 샘플, 저장 후)
10ns :[  DRV  ] addr = 16 ,we = 1, wdata = 212 , rdata=   x
16ns :[  MON  ] addr = 16 ,we = 1, wdata = 212 , rdata= 212
```

쓰기 사이클에서 v2의 rdata는 X지만, v3는 1ns 뒤에 샘플하므로 방금 쓴 212가 보인다. 6-2절의 설명을 실제 로그로 확인한 결과다.

7-5. 실행 결과와 보완할 점 확인

최종 코드(v2, v3 모두 300회 실행)를 수정 없이 Vivado 2020.2 xsim으로 시드 3개씩 실행했다. 전부 FAIL 0이지만, **PASS의 내용**은 더 살펴볼 부분이 있다.

TB	시드	전체	쓰기	읽기	PASS	FAIL	X를 읽은 횟수	we 최장 연속	wdata 최댓값
v2	1	300	162	138	138	0	43 / 138	6	254
v2	2	300	151	149	149	0	58 / 149	7	255
v2	3	300	154	146	146	0	42 / 146	8	255
v3	1	300	154	146	300	0	57 / 146	8	252
v3	2	300	134	166	300	0	69 / 166	8	255
v3	3	300	142	158	300	0	49 / 158	7	255

확인 사항	현재 상태	보완 방향
① **v3 쓰기 판정 대상** 236행	**수정 완료.** Monitor의 wdata가 아니라 rdata와 비교하도록 바뀜, `#1` 뒤 DUT 출력까지 판정한다	현재 `==`를 사용하며 rdata가 X여도 FAIL로 처리되어 동작에는 문제없음. 읽기 판정과 맞추려면 `===`로 통일 가능
② **읽기 PASS 중 X 비율**	**개선됨.** 300회로 늘려 대부분이던 X 읽기가 줄었지만, 아직 읽기의 29~42%가 미기록 주소의 X === X	실행 횟수를 더 늘리거나 주소 범위를 좁히면 "쓴 값을 다시 읽는" 경우가 늘어남
③ **`randc` 순환 초기화**	매번 `tr = new()`로 객체를 새로 만들어 순환 상태가 유지되지 않는다. 표의 we 최장 연속이 모두 2를 넘음	5:5가 필요하면 같은 객체를 재사용해 randomize. 수업의 "정확히 5:5가 안 되더라"의 원인일 수 있음. 카운터의 run_stop도 동일
④ **wdata 범위** 37~39행	**수정 완료.** v2, v3 모두 `inside {[0:255]}`로 변경	표의 wdata 최댓값이 252~255로, 상위 비트까지 검증됨
⑤ **트랜잭션 공유** 83~84행	같은 `tr` 핸들을 두 메일박스에 전달	수업 권고대로 복사본(TR2)을 따로 만들어 전달
⑥ **카운터 hold 길이**	`tb_counter_rsm_v2`는 실제 hold-1 클럭만 유지 (1-7절)	hold 전체가 유지되도록 반복 횟수와 이벤트 시점을 재검토

8장 · 수업 질문으로 복습하기

강사가 수업 중 던진 질문과 논의를 모았다. 시험과 면접 설명 연습용으로 답을 가리고 말해 보면 좋다.

질문	답
3회 이상 연속을 보장하려면 무엇이 필요한가?	범위 제약이 걸린 반복 횟수 랜덤 변수
반복은 왜 Driver에서 하는가?	Gen–Drv가 동기화되어 있어 메시지 1개에 횟수를 실으면 Generator 조건이 필요 없음
`randc`에 `dist`를 못 쓰는 이유는?	모든 값이 한 번씩 나와야 하므로 확률을 조작할 수 없음
드라이브는 왜 하강 엣지인가?	상승 엣지 전에 입력을 안정시켜 레이스를 피함
조합 출력 RAM은 언제 rdata가 바뀌는가?	주소 또는 메모리 값이 바뀌는 즉시
Expected를 큐가 아니라 배열에 저장하는 이유는?	주소로 바로 접근해야 함. 큐는 FIFO 성격
Expected 배열이 `byte`면 안 되는 이유는?	2-state라 미기록이 0이 되어, 0을 쓴 경우와 구분 불가
왜 `==`가 아니라 `===`인가?	X를 포함한 비교는 케이스 등가로만 참/거짓이 확정됨
we=1일 때도 왜 모니터링하는가?	we가 랜덤이라 Scoreboard가 저장 시점을 알아야 함
상승 엣지 순간에 쓰기 결과를 볼 수 있는가?	저장 전이라 불가. 지연 후 또는 다음 하강 엣지에 확인
드라이브 직후 이벤트를 날리면 왜 위험한가?	같은 시각이라 수신 측이 대기 전이면 이벤트를 놓침

질문	답
그 레이스를 피하는 방법은?	`#1` 지연, `->>`, `wait(ev.triggered)`, 구조 변경
Scoreboard→Generator 이벤트가 없으면?	Generator가 혼자 계속 돌아 버림
같은 트랜잭션을 두 곳에 보낼 때 주의점은?	핸들 공유. 복사본을 만들어 보내는 편이 안전
순차 출력 RAM에 같은 시나리오를 쓸 수 있는가?	불가. rdata가 상승 엣지 이후에 나오므로 타이밍 재설계

9장 · 숙제와 할 일

항목	출처 / 상태
☐ 카운터 TB 시나리오·흐름도 정리, mode까지 확장, 제어 방식 설명 준비	1교시 / hold 길이는 7-5절 ⑥ 참고
☐ RAM TB 블록 다이어그램 직접 작성	3교시
☐ gen→scb 메일박스 + 매 사이클 판정 구조 완성 후 이전 구조와 비교	6교시 숙제 / `tb_ram_v3.sv` 작성 완료, 7-5절 ①의 쓰기 판정 수정 반영
☐ 순차 출력 RAM의 타이밍 계획·검증	2·4교시 언급 / 아직 조합 출력만 진행
☐ SV로 FIFO 검증 준비	2교시 일정
☐ 11/16 핸드라이팅 시험 대비	2-3절, 8장 참고
☐ 11/23 UVM 발표 주제 확정	2-1절 / 강사 확인 필요

10장 · 녹음에서 확인이 필요한 부분

녹음 변환 오류가 많아 아래는 문맥으로 해석했다. 일정과 시험 관련 항목은 강사에게 확인하는 것을 권장한다.

녹음 표기	정리본 해석	상태
만진 카운터	10,000진 카운터	문맥상 유력
월드, 올드, 골드, 폴드	hold (반복 횟수 변수)	코드와 일치
FMD	FND	문맥상 유력
시답시토리, 시나 스토리	Synopsys 툴 (* 표시 항목)	추정
3이 아이	RV32I	문맥상 유력
에큐리 버스	RISC-V에 붙일 버스 프로토콜	**이름 불명, 확인 필요**
오회점 트리플럽	플립플롭 (종류 미상)	추정
백관식 한두 문제	시험의 개념 문제 형식	**객관식/주관식 확인 필요**
프 펙션, 앰플7	시험 범위 언급으로 보이나 복원 불가	**확인 필요**

녹음 표기	정리본 해석	상태
메모리 이런 거 안 배운 거 같고	메모리 설계는 시험에서 제외로 보임	**확인 필요**
발표 주제 최종안	"2가지 안을 놓고 가"까지만 확인	**미확정**
3교시 말미(197행), 6교시 끝(293~296행)	학생과의 개별 대화와 잡음은 복원 불가하여 제외	제외